JPA pagination

학습목표

JPA pagination 기능 구현

목차

1. 배경지식 2
    1) JPA Repository의 페이지 조회 기본 메소드 2
    2) JPA 로그 3
2. jpa3 프로젝트 4
    1) 프로젝트 생성 4
    2) build.gradle 파일 수정 6
    3) application.properties 6
    4) Department 엔터티 클래스 7
    5) Student 엔터티 클래스 7
    6) DepartmentRepository 클래스 8
    7) StudentRepository 클래스 8
    8) StudentEdit 모델 클래스 9
    9) Pagination.java 모델 10
    10) Student Controller 구현 11
    11) common.css 14
    12) common.js 15
    13) student/list 뷰 구현 16
    14) student/edit.html 18
    15) 실행 19
3. 연습문제 20

1.배경지식

1) JPA Repository의 페이지 조회 기본 메소드

StudentRepository 에는 아래 메소드들이 자동으로 구현된다.
즉 아래 메소드들은 JpaRepository가 기본으로 제공하는 메소드이다.

public List<Student> findAll();
student 테이블의 모든 레코드를 조회해서 리턴한다.

public Page<Student> findAll(Pageable pageable);
student 테이블에서 한 페이지의 레코드들만 조회하여 리턴한다.
Pageable 객체에 담겨있는 페이지 번호와 정렬순서로, 한 페이지의 레코드들만 조회 한다.
이 메소드도 자동으로 구현되는 기본 메소드이다.

이 메소드를 사용하는 방법은 다음과 같다.

```
PageRequest pageRequest = PageRequest.of(페이지번호, 페이지크기, 정렬순서방향, 정렬기준속성명...)
studentRepository.findAll(pageRequest);
```

PageRequest 클래스는 Pageable 인터페이스의 자식 클래스이다.

PageRequest.of(페이지번호, 페이지크기, 정렬순서, 정렬기준속성명...)
PageRequest 클래스의 of 메소드는 static 메소드이다.
이 of 메소드는 **PageRequest** 객체를 생성하여 리턴한다.
페이지번호: 조회할 페이지 번호, 페이지 번호는 0 부터 시작한다.
페이지크기: 한 페이지의 레코드 수
정렬순서방향: Sort.Direction.ASC, Sort.Direction.DESC 둘 중의 하나
정렬기준속성명: 정렬 기준이될 엔터티 속성 이름을 문자열로

사용 예:

(1) `PageRequest pageRequest = PageRequest.of(0, 10, Sort.Direction.ASC, "id");`
PageRequest 객체를 생성한다.
이 객체에는 다음과 같은 정보가 담겨있다.
(id 속성 오름차순으로 정렬하여, 첫 페이지 10 개의 레코드만 조회하라)

(2) `Page<Student> page = studentRepository.findAll(pageRequest);`
pageRequest객체에 담겨있는 정렬순서와 페이지 번호를 적용하여 student 테이블을 조회한다.
그 페이지의 레코드들만 조회되어 리턴된다.

(3) `page.getTotalElements()`
student 테이블의 전체 레코드 수를 리턴한다.

(4) `page.getContent();`
조회 결과 레코드를 리턴한다. List<Student>

2) JPA 로그

디버깅 할 때, JPA 에 의해서 생성되는 SQL 명령과 파라미터를 확인해 보고 싶은 경우가 있다.

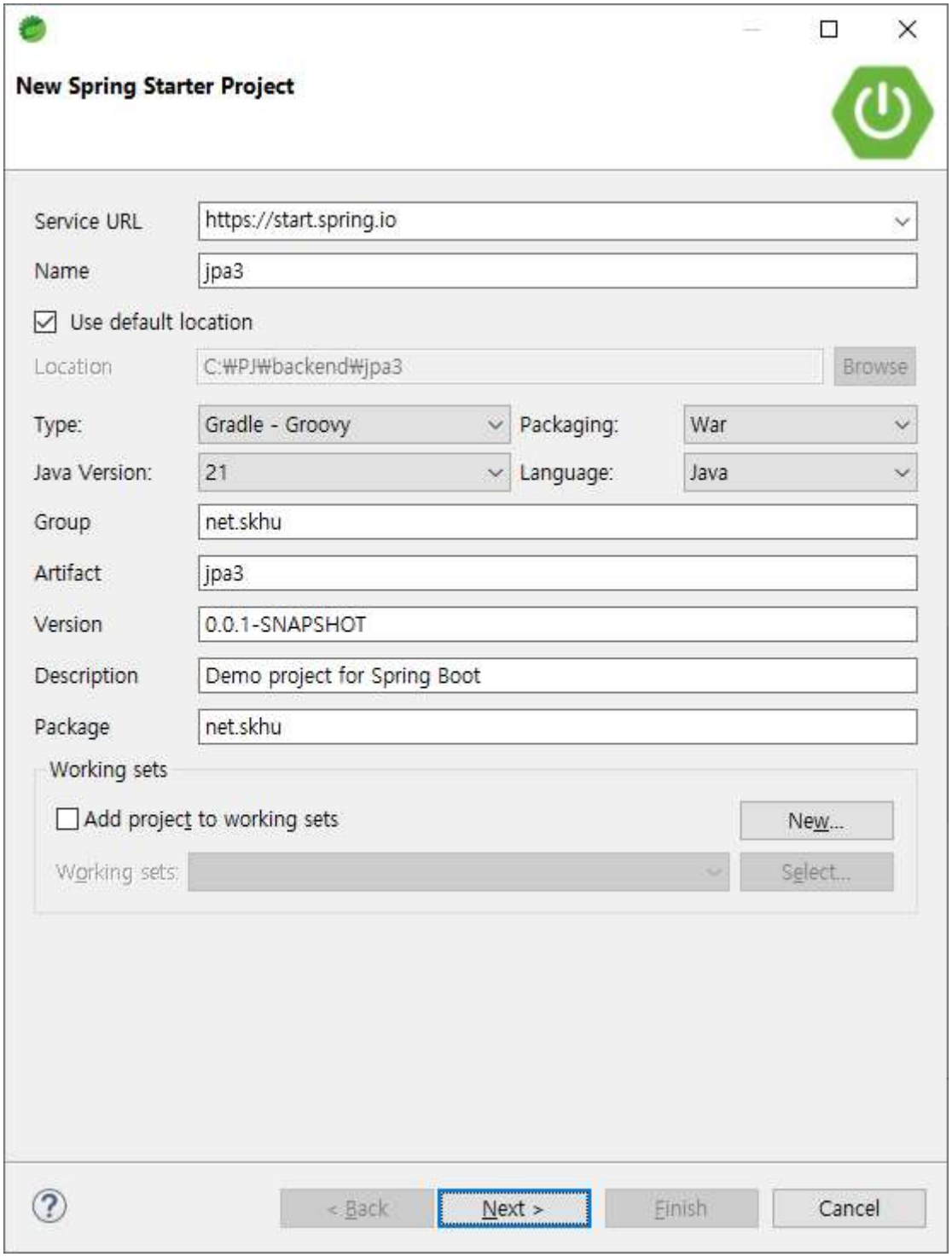
application.properties 파일

```
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
```

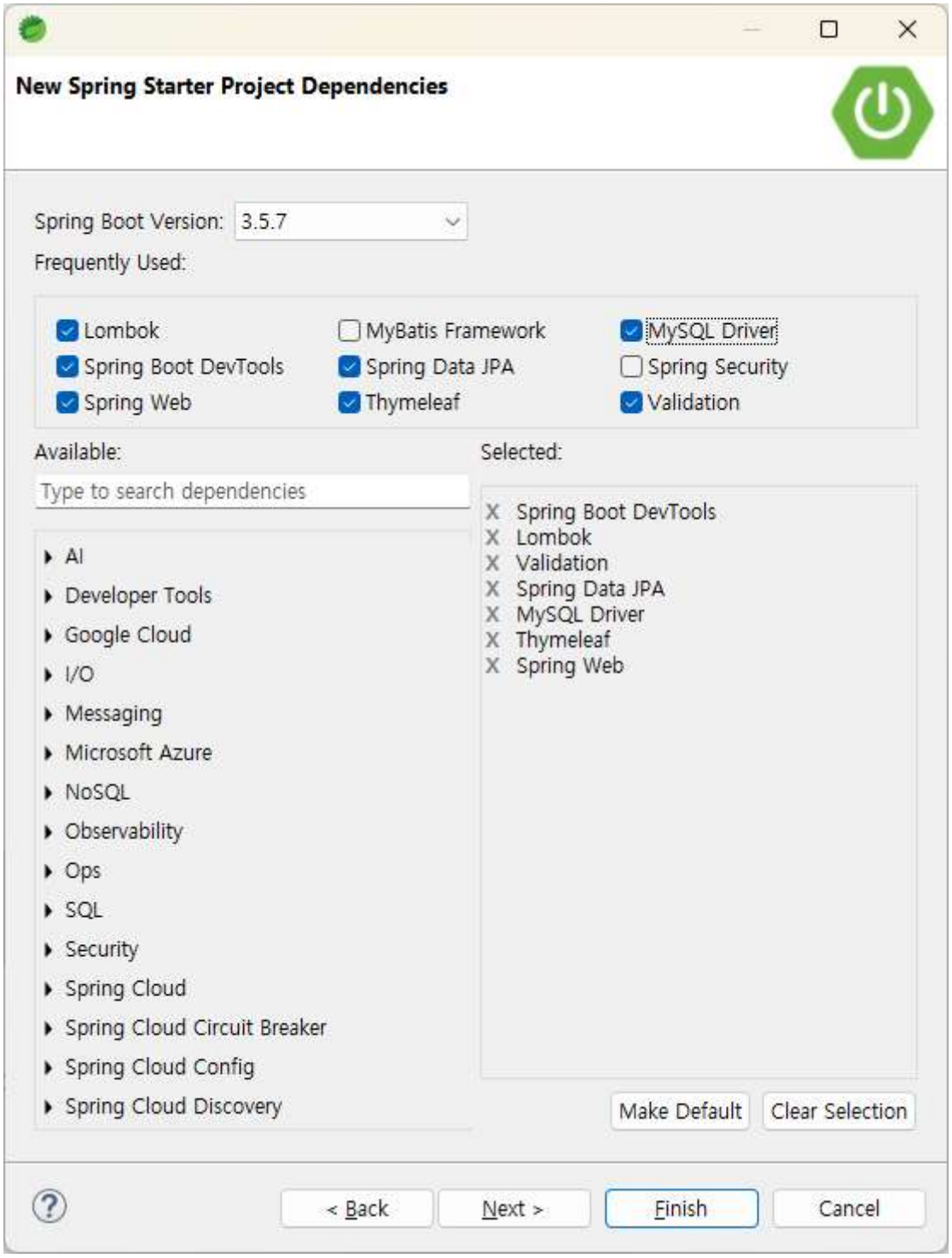
spring data JPA에 의해서 생성되고 실행되는 SQL 명령들을
이클립스 console 창에서 확인할 수 있다.

2. jpa3 프로젝트

1) 프로젝트 생성



Name	jpa3
Type	Gradle - Groovy
Java Version	21
Package	net.skhu




```
Spring Boot DevTools
Lombok
Validation
Spring Data JPA
MySQL Driver
Thymeleaf
Spring Web
```

2) build.gradle 파일 수정

```
dependencies {
  ... 생략 ...
    implementation 'org.modelmapper:modelmapper:3.2.5'
}
```

gradle 프로젝트의 설정 파일인 build.gradle 파일에
  modelmapper maven dependency 한 줄을 추가

build.gradle 파일을 수정한 후에는 반드시 Refresh Gradle Project 해야 한다.

3) application.properties

src/main/resources/application.properties

```
spring.application.name=jpa3
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/student2?useUnicode=yes&characterEncoding=UTF-8&allowMultiQueries=true&serverTimezone=Asia/Seoul
spring.datasource.username=user1
spring.datasource.password=skhuA+4.5
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.orm.jdbc.bind=TRACE
logging.level.net.skhu=DEBUG
server.port=8088
```

4) Department 엔터티 클래스

src/main/java/net/skhu/entity/Department.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Data;

@Data
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String name;
    String shortName;
    String phone;
}
```

5) Student 엔터티 클래스

src/main/java/net/skhu/entity/Student.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import lombok.Data;

@Data
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String studentNo;
    String name;
    String phone;
    String sex;
    String email;

    @ManyToOne
    @JoinColumn(name = "departmentId")
    Department department;
}
```

6) DepartmentRepository 클래스

src/main/java/net/skhu/repository/DepartmentRepository.java

```
1  package net.skhu.repository;
2
3  import org.springframework.data.jpa.repository.JpaRepository;
4  import net.skhu.entity.Department;
5
6  public interface DepartmentRepository extends JpaRepository<Department, Integer>  {
7
8  }
```

7) StudentRepository 클래스

src/main/java/net/skhu/repository/StudentRepository.java

```
1  package net.skhu.repository;
2
3  import org.springframework.data.jpa.repository.JpaRepository;
4  import net.skhu.entity.Student;
5
6  public interface StudentRepository extends JpaRepository<Student, Integer>  {
7
8  }
```

8) StudentEdit 모델 클래스

src/main/java/net/skhu/model/StudentEdit.java

```java
package net.skhu.model;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.Data;

@Data
public class StudentEdit {
    int id;

    @NotEmpty @NotBlank
    @Size(min=8, max=12)
    String studentNo;

    @NotEmpty @NotBlank
    @Size(min=2, max=20)
    String name;

    @NotEmpty @NotBlank
    @Pattern(regexp="010-[0-9]{3,4}-[0-9]{4}",
             message="휴대폰 번호를 입력하세요 예:010-000-0000")
    String phone;

    @NotEmpty @Email
    String email;

    @NotEmpty @NotBlank
    @Pattern(regexp="남|여", message="남, 여 중 하나를 입력하세요.")
    String sex;

    @Min(value=1, message="학과를 선택하세요.")
    int departmentId;
}
```

9) Pagination.java 모델

src/main/java/net/skhu/model/Pagination.java

```java
package net.skhu.model;

import lombok.Data;

@Data
public class Pagination {
    int pg = 1;         // 현재 페이지 번호
    int sz = 15;        // 페이지 당 레코드 수
    int recordCount;    // 전체 레코드 수

    public int getFirstRecordIndex() {
        return (pg - 1) * sz;
    }

    public String getQueryString() {
        return String.format("pg=%d&sz=%d", pg, sz);
    }
}
```

페이지 단위 조회 기능을 구현하기 위해서,
액션 메소드와 뷰 사이에 전달되어야 하는 정보를 담은 객체이다.

10) Student Controller 구현

src/main/java/net/skhu/controller/StudentController.java

```java
package net.skhu.controller;

import java.util.List;
import java.util.Optional;
import org.modelmapper.ModelMapper;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.server.ResponseStatusException;
import jakarta.validation.Valid;
import lombok.extern.slf4j.Slf4j;
import net.skhu.entity.Student;
import net.skhu.model.Pagination;
import net.skhu.model.StudentEdit;
import net.skhu.repository.DepartmentRepository;
import net.skhu.repository.StudentRepository;

@Controller
@RequestMapping("student")
@Slf4j
public class StudentController {

    @Autowired StudentRepository studentRepository;
    @Autowired DepartmentRepository departmentRepository;
    ModelMapper modelMapper = new ModelMapper();

    @GetMapping("list")
    public String list(Model model, Pagination pagination) {
        Page<Student> page = studentRepository.findAll(PageRequest.of(pagination.getPg() - 1,
                pagination.getSz(), Sort.Direction.ASC, "id"));
        pagination.setRecordCount((int)page.getTotalElements());
        model.addAttribute("students", page.getContent());
        return "student/list";
    }

    @GetMapping("edit")
    public String edit(Model model,  Pagination pagination, @RequestParam("id") int id) {
        Optional<Student> student = studentRepository.findById(id);
        if (student.isEmpty())
            throw new ResponseStatusException(HttpStatus.NOT_FOUND);
        StudentEdit studentEdit = modelMapper.map(student.get(), StudentEdit.class);
        model.addAttribute("studentEdit", studentEdit);
        model.addAttribute("departments", departmentRepository.findAll());
        return "student/edit";
    }

    @PostMapping(value="edit", params="cmd=save")
    public String edit(Model model, Pagination pagination,
            @Valid StudentEdit studentEdit, BindingResult bindingResult) {
        try {
            log.debug(studentEdit.toString());
            if (bindingResult.hasErrors())
                throw new Exception("저장할 수 없습니다");
            Student student = modelMapper.map(studentEdit, Student.class);
            studentRepository.save(student);
            return "redirect:list?" + pagination.getQueryString();
        }
        catch (Exception ex) {
            bindingResult.reject("", null, ex.getMessage());
            model.addAttribute("departments", departmentRepository.findAll());
            return "student/edit";
        }
    }

    @PostMapping(value="edit", params="cmd=delete")
    public String delete(Model model, Pagination pagination,
            @Valid StudentEdit studentEdit, BindingResult bindingResult) {
        try {
            studentRepository.deleteById(studentEdit.getId());
            return "redirect:list?" + pagination.getQueryString();
        }
        catch (Exception ex) {
            bindingResult.reject("", null, ex.getMessage());
            model.addAttribute("departments", departmentRepository.findAll());
```

```
84              return "student/edit";
85          }
86      }
87
88      @GetMapping("create")
89      public String create(Model model, Pagination pagination) {
90          StudentEdit studentEdit = new StudentEdit();
91          model.addAttribute("studentEdit", studentEdit);
92          model.addAttribute("departments", departmentRepository.findAll());
93          return "student/edit";
94      }
95
96      @PostMapping("create")
97      public String create(Model model, Pagination pagination,
98              @Valid StudentEdit studentEdit, BindingResult bindingResult) {
99          try {
100             log.debug(studentEdit.toString());
101             if (bindingResult.hasErrors())
102                 throw new Exception("저장할 수 없습니다");
103             Student student = modelMapper.map(studentEdit, Student.class);
104             studentRepository.save(student);
105             int lastPage = (int)Math.ceil((double)studentRepository.count() / pagination.getSz());
106             pagination.setPg(lastPage);
107             return "redirect:list?" + pagination.getQueryString();
108         }
109         catch (Exception ex) {
110             bindingResult.reject("", null, ex.getMessage());
111             model.addAttribute("departments", departmentRepository.findAll());
112             return "student/edit";
113         }
114     }
115 }
```

노란색으로 칠한 부분만 mybatis pagination 구현과 다르다.

```
@GetMapping("list")
public String list(Model model, Pagination pagination) {
    Page<Student> page = studentRepository.findAll(PageRequest.of(pagination.getPg() - 1,
            pagination.getSz(), Sort.Direction.ASC, "id"));
    pagination.setRecordCount((int)page.getTotalElements());
    model.addAttribute("students", page.getContent());
    return "student/list";
}
```

페이지 단위 조회 기능을 구현한 액션 메소드이다.
이 메소드는 Pagination 모델 객체에 담겨있는, 페이지 정보에 따라,
현재 페이지의 레코드들을 조회하여 출력한다.

StudentRepository에 자동으로 구현된 Page<Student> findAll(Pageable pageable);
기본 메소드를 사용하여 구현하였다.

```
Page<Student> page = studentRepository.findAll(PageRequest.of(
        pagination.getPg() - 1,  // 현재 페이지 번호.
                                 //   JPA pagination 번호는 0부터 시작하기 때문에 - 1 했다.
        pagination.getSz(),      // 페이지의 레코드 수
        Sort.Direction.ASC, "id"));     // 정렬 순서
```

StudentRepository에 자동으로 구현된 findAll 메소드를 호출하여
현재 페이지 번호에 해당하는 레코드들만 조회된다.

findAll 메소드 Page<Student> page 객체를 리턴한다.
이 객체에 조화 결과 레코드들과
전체 레코드 수가 들어있다.

```
pagination.setRecordCount((int)page.getTotalElements());
     // page.getTotalElements() 값은 전체 레코드 수이다.
     // 전체 레코드 수를 Pagination 객체에 채운다.

model.addAttribute("students", page.getContent());
     // page.getContent() 값은 조회된 학생 목록이다.
     // List<Student> 타입의 학생 목록이고, 현재 페이지에 해당하는 레코드들만 조회되었다.
     // 이 학생 목록을 뷰(student/list.html)에 전달한다. (model attribute)
```

11) common.css

src/main/resources/static/common.css

```
div.container { width: 800px; margin: 10px auto; font-size: 11pt; }
.shadow { box-shadow: 2px 2px 5px gray; }

.btn { padding: 0.4em 1em; border: 1px solid gray;
       border-radius: 0.5em; background: linear-gradient(#fff, #ddd);
       text-decoration: none; color: black;
       display: inline-block; }
.btn:active {
      -ms-transform: translateY(2px);
      -webkit-transform: translateY(2px);
      transform: translateY(2px);
      background: #ccc; }

table.list { border-collapse: collapse; width: 100%; }
table.list td { padding: 4px; border: 1px solid gray; }
table.list th { padding: 4px; border: 1px solid gray;  background-color: #eee; }

div.row { margin-bottom: 0.8em; }
div.row > label:nth-child(1) { display: inline-block; min-width: 4em; text-align: right; }

input[type=text], input[type=number] { padding: 0.3em; }
select { padding: 0.3em; }
h1 { margin: 0.5em 0; }
hr { margin: 1em 0; }
.error { color: red; padding: 0.2em; margin-left: 1em; }

div.pagination table { border-collapse: collapse; margin-top: 1em; }
div.pagination table td { border: 1px solid #ccc; text-align: center; }
div.pagination table td.active { background-color: #48f; }
div.pagination table a { text-decoration: none; color: blue; display: inline-block;
                         width: 25px; padding: 6px; }
div.pagination table td.active a { color: white; }
```

12) common.js

src/main/resources/static/common.js

```
function onClickHref(event) {
    let url = event.currentTarget.getAttribute("href");
    location.href = url;
}

function createPageTd(no, label, url) {
    let td = document.createElement("td");
    let a = document.createElement("a");
    a.appendChild(document.createTextNode(label));
    a.setAttribute("href", url.replace(/pg=[0-9]+/, "pg=" + no));
    td.appendChild(a);
    return td;
}

function pagination() {
    let div = document.getElementsByClassName("pagination")[0];
    if (div == undefined) return;
    let recordCount = parseInt(div.getAttribute("data-record-count"));
    let pageSize = parseInt(div.getAttribute("data-page-size"));
    let currentPage = parseInt(div.getAttribute("data-current-page"));
    let pageCount = Math.ceil(recordCount / pageSize);
    if (currentPage > pageCount) currentPage = pageCount;
    let url = location.href;
    if (url.indexOf("pg=") < 0)
        url = url + (url.indexOf("?") < 0 ? "?pg=1" : "&pg=1");
    let table = document.createElement("table");
    let tr = document.createElement("tr");
    let baseNo = Math.floor((currentPage - 1) / 10) * 10;
    if (baseNo > 0) tr.appendChild(createPageTd(baseNo, "<", url));
    for (let i = 1; i <= 10; ++i) {
        let no = baseNo + i;
        if (no > pageCount) break;
        let td = createPageTd(no, String(no), url)
        if (no == currentPage) td.classList.add("active");
        tr.appendChild(td);
    }
    let no = baseNo + 11;
    if (no <= pageCount) tr.appendChild(createPageTd(no, ">", url));
    table.appendChild(tr);
    div.appendChild(table);
}

window.onload = pagination;
```

13) student/list 뷰 구현

src/main/resources/templates/student/list.html

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" th:href="@{/common.css}" />
  <script type="text/javascript" th:src="@{/common.js}"></script>
  <style>
    form { margin-bottom: 4px; }
    a.btn { float:right; margin-top: -37px; }
  </style>
</head>
<body>
<div class="container">
  <h1>학생목록</h1>
  <a th:href="'create?' + ${pagination.queryString}" class="btn">학생 등록</a>
  <table class="list">
    <thead>
      <tr>
        <th>id</th>
        <th>학번</th>
        <th>이름</th>
        <th>학과</th>
        <th>전화</th>
        <th>성별</th>
        <th>이메일</th>
      </tr>
    </thead>
    <tbody>
      <tr th:each="st : ${students}">
        <td th:text="${st.id}"></td>
        <td>
          <a th:text="${ st.studentNo }"
             th:href="'edit?id=' + ${st.id} + '&' + ${pagination.queryString}"></a>
        </td>
        <td th:text="${st.name}"></td>
        <td th:text="${st.department.name}"></td>
        <td th:text="${st.phone}"></td>
        <td th:text="${st.sex}"></td>
        <td th:text="${st.email}"></td>
      </tr>
    </tbody>
  </table>
  <div class="pagination"
    th:data-record-count="${pagination.recordCount}"
    th:data-page-size="${pagination.sz}"
    th:data-current-page="${pagination.pg}"></div>
</div>
</body>
</html>
```

노란색으로 칠한 부분만 mybatis pagination 구현과 다르다.

"${ st.department.name }"
   list.html 뷰에 전달된 학생 목록에서 학생 객체는
      Student 엔터티 클래스 객체이다.
      Student 엔터티 클래스에는 departmentName 속성이 없고
      department 속성이 있다.
      이 department 속성은 Department 객체이고
      Department 객체에는 name 속성이 있다.

```
<a th:href="'create?' + ${pagination.queryString}" class="btn">
  예를 들어서 pagination 모델 객체의 queryString 속성값이 "pg=1&sz=15" 이라면,
  위 thymeleaf 소스코드는 다음과 같은 html 태그를 출력한다.
  <a href="create?pg=1&sz=15" class="btn">
```

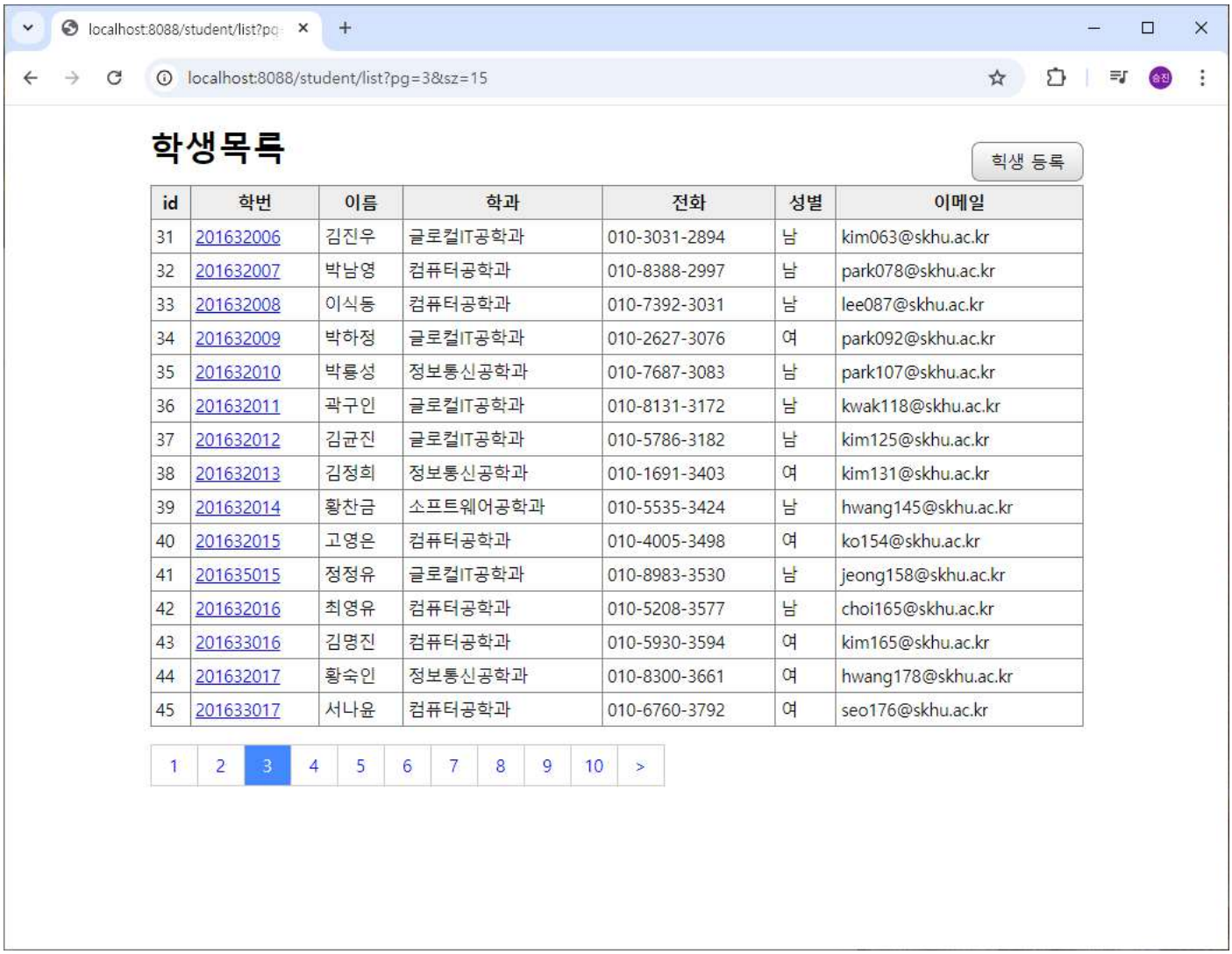
```
<a th:text="${ st.studentNo }"
   th:href="'edit?id=' + ${st.id} + '&' + ${pagination.queryString}"></a>
  예를 들어서 pagination 모델 객체의 queryString 속성값이 "pg=1&sz=15" 이고,
  st 객체의 studentNo 속성값이 "201514099"이고 id 속성값이 3 이라면,
  위 thymeleaf 소스코드는 다음과 같은 html 태그를 출력한다.
  <a href="edit?id=3&pg=1&sz=15" class="btn">201514099</a>
```

14) student/edit.html

src/main/resources/templates/student/edit.html

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" th:href="@{/common.css}" />
  <style>
    form { width: 500px; padding: 1em 3em; }
    button:nth-of-type(1) { margin-left: 1em; }
  </style>
</head>
<body>
<div class="container">
 <form method="post" th:object="${studentEdit}" class="shadow">
    <h1 th:if="${studentEdit.id > 0}">학생 수정</h1>
    <h1 th:if="${studentEdit.id == 0}">학생 등록</h1>
    <div class="row">
      <label>학번:</label>
      <input type="text" th:field="*{studentNo}" />
      <div class="error" th:errors="*{studentNo}"></div>
    </div>
    <div class="row">
      <label>이름:</label>
      <input type="text" th:field="*{name}" />
      <div class="error" th:errors="*{name}"></div>
    </div>
    <div class="row">
      <label>학과:</label>
      <select th:field="*{departmentId}">
        <option value="0">학과를 선택하세요</option>
        <option th:each="d : ${departments}"
                th:value="${d.id}" th:text="${d.name}"></option>
      </select>
      <div class="error" th:errors="*{departmentId}"></div>
    </div>
    <div class="row">
      <label>성별:</label>
      <label><input type="radio" th:field="*{sex}" value="남" />남자</label>
      <label><input type="radio" th:field="*{sex}" value="여" />여자</label>
      <div class="error" th:errors="*{sex}"></div>
    </div>
    <div class="row">
      <label>전화:</label>
      <input type="text" th:field="*{phone}" />
      <div class="error" th:errors="*{phone}"></div>
    </div>
    <div class="row">
      <label>메일:</label>
      <input type="text" th:field="*{email}" />
      <div class="error" th:errors="*{email}"></div>
    </div>
    <hr />
    <button type="submit" class="btn"
             name="cmd" value="save">저장</button>
    <button type="submit" class="btn"
             th:if="${ studentEdit.id > 0 }"
             onclick="return confirm('삭제하시겠습니까?')"
             name="cmd" value="delete">삭제</button>
    <a th:href="'list?' + ${pagination.queryString}" class="btn">목록으로</a>
    <span class="error" th:errors="${studentEdit}"></span>
  </form>
</div>
</body>
</html>
```

15) 실행



id	학번	이름	학과	전화	성별	이메일
31	201632006	김진우	글로컬IT공학과	010-3031-2894	남	kim063@skhu.ac.kr
32	201632007	박남영	컴퓨터공학과	010-8388-2997	남	park078@skhu.ac.kr
33	201632008	이식등	컴퓨터공학과	010-7392-3031	남	lee087@skhu.ac.kr
34	201632009	박하정	글로컬IT공학과	010-2627-3076	여	park092@skhu.ac.kr
35	201632010	박룡성	정보통신공학과	010-7687-3083	남	park107@skhu.ac.kr
36	201632011	곽구인	글로컬IT공학과	010-8131-3172	남	kwak118@skhu.ac.kr
37	201632012	김균진	글로컬IT공학과	010-5786-3182	남	kim125@skhu.ac.kr
38	201632013	김정희	정보통신공학과	010-1691-3403	여	kim131@skhu.ac.kr
39	201632014	황찬금	소프트웨어공학과	010-5535-3424	남	hwang145@skhu.ac.kr
40	201632015	고영은	컴퓨터공학과	010-4005-3498	여	ko154@skhu.ac.kr
41	201635015	정정유	글로컬IT공학과	010-8983-3530	남	jeong158@skhu.ac.kr
42	201632016	최영유	컴퓨터공학과	010-5208-3577	남	choi165@skhu.ac.kr
43	201633016	김명진	컴퓨터공학과	010-5930-3594	여	kim165@skhu.ac.kr
44	201632017	황숙인	정보통신공학과	010-8300-3661	여	hwang178@skhu.ac.kr
45	201633017	서나윤	컴퓨터공학과	010-6760-3792	여	seo176@skhu.ac.kr

3. 연습문제

book2 데이터베이스의 book 테이블에 대해서 pagination을 구현하라.